

Process note – Tamar Buznah

Tool Used: Claude Code

Key Prompts

My goal was to use AI to rapidly prototype a clean, zero-dependency Python CLI tool while maintaining strict architectural control over the solution. Key prompts included:

Core Logic: *"Write a zero-dependency Python 3.11 script to parse Jenkins console logs. Extract failing stages, classify root causes using regex patterns, and output a structured Markdown report."*

Multi-Format Support: *"Update triage.py to support --format md|text|json and -o/--output using a dispatch dictionary (RENDERERS) and argparse."*

What the AI Got Wrong & What I Corrected

AI Mistake: The initial AI code picked generic Jenkins pipeline wrapper lines (e.g., "ERROR: script returned exit code 1") as the primary failure reason.

My Fix: Introduced a NOISE_RE regex filter to explicitly skip Jenkins execution boilerplate and search for the actual underlying exception (e.g., AssertionError, ResolutionImpossible, GitException).

AI Mistake: The AI originally suggested nested `if-elif` blocks and extra helper abstractions for formatting output.

My Fix: Directing the AI to use a clean Dictionary Dispatch pattern (`RENDERERS = {"md": ..., "text": ..., "json": ...}`), resulting in a modular, readable single file script